

ananasiki/auto-reviewer

Python

Оценено 28 сентября 2026 г.

Данных собрано 53%



БАЛЛ ЗДОРОВЬЯ

Проекту нужна серьёзная работа.

ЧТО ЭТО · О ПРОЕКТЕ

Auto-Reviewer — система для автоматического ревью студенческого кода с AI-анализом и участием преподавателей. В репозитории есть REST API и обработчик GitHub webhooks на Python, сервис очередей для анализа кода и веб-панель на React; система использует PostgreSQL и RabbitMQ.

- AI-ревью с GPT-4o-mini и образовательной обратной связью
- AST-анализ, метрики качества и поиск опасных функций
- GitHub App и автоматические комментарии к pull request
- Backend API на Litestar и consumer на FastStream
- Веб-панель на React и Vite

● Безопасность



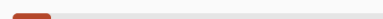
● Код

52



● Активность

10



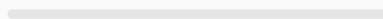
● Документация

72



● CI/CD

0



● Задачи



Штрафы

-8 Файл LICENSE не найден

Категории и метрики

● **Безопасность** вес 20%

Critical-находки AppSec

Нет данных SourceCraft AppSec

High-находки AppSec

Нет данных SourceCraft AppSec

Medium-находки AppSec

Нет данных SourceCraft AppSec

Секреты в репозитории (AppSec)

Нет данных SourceCraft AppSec

● **Код** вес 20%

52

Ревью кода моделью

52

В выборке есть понятные названия и разделение по доменам, но крупный AST-модуль и дублирование GitHub-аутентификации снижают структурную оценку. Обработка ошибок местами скрывает отказ анализа как безопасный результат, а видимые тесты не демонстрируют проверок поведения.

Активность вес 15%**10****Коммитов за 90 дней**

Коммитов за 90 дней: 0

Активных авторов

Активных авторов за 90 дней: 0

Свежесть последнего коммита

Последний коммит: 381 дн. назад

Bus factor

Не удалось посчитать долю кода топ-автора

Релизы и теги

Релизов и тегов нет

Поток pull request

Pull request: 6

Документация вес 15%**72****Документация по рубрике модели**

README хорошо структурирован: описывает возможности, архитектуру, быстрый запуск и общий рабочий процесс. Установка и API-документация неполны: не перечислены требования и подробная настройка окружения, а публичные эндпоинты почти не описаны; ссылки на дополнительные руководства частично компенсируют этот пробел.

CI/CD вес 15%**0****Конфиг CI**

Конфига CI нет

Тесты в пайплайне

Конфига CI нет — проверять нечего

Линтер в пайплайне

Конфига CI нет — проверять нечего

Проверка pull request

Конфига CI нет — проверять нечего

Успешные прогоны CI

Прогоны CI видны только участникам — в публичный балл не входят

● Задачи вес 15%

Закрытие задач

Трекером задач не пользуются

Заброшенные задачи

Трекером задач не пользуются

Скорость реакции на задачи

Трекером задач не пользуются

Что сделать

1 Настройте CI +23

Сейчас: Конфига CI нет

Создайте `.sourcecraft/ci.yaml`: запуск на `push` и `pull_request`, шаги — `pip install -r requirements.txt`, `ruff check .`, `pytest`. Pulse может подготовить этот файл в `pull request`.

Усилия: день

2 Зафиксируйте активность свежим коммитом +5

Сейчас: Последний коммит: 381 дн. назад

Проект давно не обновлялся. Если он жив — обновите зависимости и выпустите версию; если нет — отметьте это в `README`.

Усилия: час

3 Выпустите релиз и поставьте тег версии +4

Сейчас: Релизов и тегов нет

Поставьте тег версии (например, `v1.0.0`) и опубликуйте релиз с описанием изменений — и заведите `CHANGELOG`.

Усилия: час

4 **Наращивайте темп разработки** **+3**

Сейчас: Коммитов за 90 дней: 0

Коммитьте чаще и мельче: одна законченная правка — один коммит. Это и ритм проекта, и удобное ревью.

Усилия: день

5 **Привлекайте ещё контрибьюторов** **+3**

Сейчас: Активных авторов за 90 дней: 0

Заведите задачи с меткой для новичков и опишите в CONTRIBUTING, как начать: так проще привлечь соавторов.

Усилия: неделя и больше

6 **Проводите изменения через pull request** **+1**

Сейчас: Pull request: 6

Вносите изменения через pull request даже в одиночку: история решений и место для проверок CI.

Усилия: день

Каких данных нет

- В репозитории нет LICENSE
- AppSec SourceCraft отдаёт результаты только участникам репозитория — в публичный балл безопасность не входит